```
from google.colab import files
uploaded = files.upload()
```

Choose files Silkworm Di…1i.yolov8.zip

Silkworm Diseases.v1i.yolov8.zip(application/x-zip-compressed) - 322106885 bytes, last modified: 15/04/2026 - 100% done

```
Saving Silkworm Diseases.v1i.yolov8.zip to Silkworm Diseases.v1i.yolov8 (1).z
```

```
import zipfile

with zipfile.ZipFile("Silkworm Diseases.v1i.yolov8 (1).zip", '
    zip_ref.extractall("/content/")
```

```
import os
print(os.listdir("/content/"))
```

```
['.config', 'valid', 'Silkworm Diseases.v1i.yolov8.zip', 'train', 'README.rob
```

```
base_path = "/content/"
```

```
import os
import shutil

# Correcting the base_path to point to the root of the extracted dataset
base_path = "/content/"

def convert(split):
    images_path = os.path.join(base_path, split, "images")
    labels_path = os.path.join(base_path, split, "labels")

    output_path = f"/content/data/{split}"

    os.makedirs(output_path + "/weed", exist_ok=True)
    os.makedirs(output_path + "/crop", exist_ok=True)

    # Check if labels_path exists before listing its contents
    if not os.path.exists(labels_path):
        print(f"Warning: Labels path not found for {split}: {labels_path}. S
        return

    for file in os.listdir(labels_path):
        with open(os.path.join(labels_path, file), "r") as f:
            content = f.read().strip()

        # Skip empty label files
        if not content:
            continue
```

```
            img_file = file.replace(".txt", ".jpg")
            img_path = os.path.join(images_path, img_file)

            if not os.path.exists(img_path):
                continue

            label = content.split()[0]

            if label == "0":
                shutil.copy(img_path, output_path + "/crop/")
            elif label == "1":
                shutil.copy(img_path, output_path + "/weed/")

    # Run conversion
    convert("train")
    convert("test")
    convert("valid")

    print("✅ Conversion Done!")
```

```
✅ Conversion Done!
```

```
import tensorflow as tf

train_data = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/data/train",
    image_size=(128,128),
    batch_size=32
)

val_data = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/data/valid",
    image_size=(128,128),
    batch_size=32
)

test_data = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/data/test",
    image_size=(128,128),
    batch_size=32
)
```

```
Found 7978 files belonging to 2 classes.
Found 499 files belonging to 2 classes.
Found 498 files belonging to 2 classes.
```

```
import tensorflow as tf
from tensorflow.keras import layers

norm = layers.Rescaling(1./255)

train_data = train_data.map(lambda x,y: (norm(x), y))
val_data = val_data.map(lambda x,y: (norm(x), y))
test_data = test_data.map(lambda x,y: (norm(x), y))
```

```
from tensorflow.keras import models

model = models.Sequential([
    layers.Conv2D(32,3,activation='relu',input_shape=(128,128,3)),
    layers.MaxPooling2D(),

    layers.Conv2D(64,3,activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128,3,activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(128,activation='relu'),
    layers.Dense(1,activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_c
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Par
conv2d (Conv2D)	(None, 126, 126, 32)	
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	
flatten (Flatten)	(None, 25088)	
dense (Dense)	(None, 128)	3,211
dense_1 (Dense)	(None, 1)	

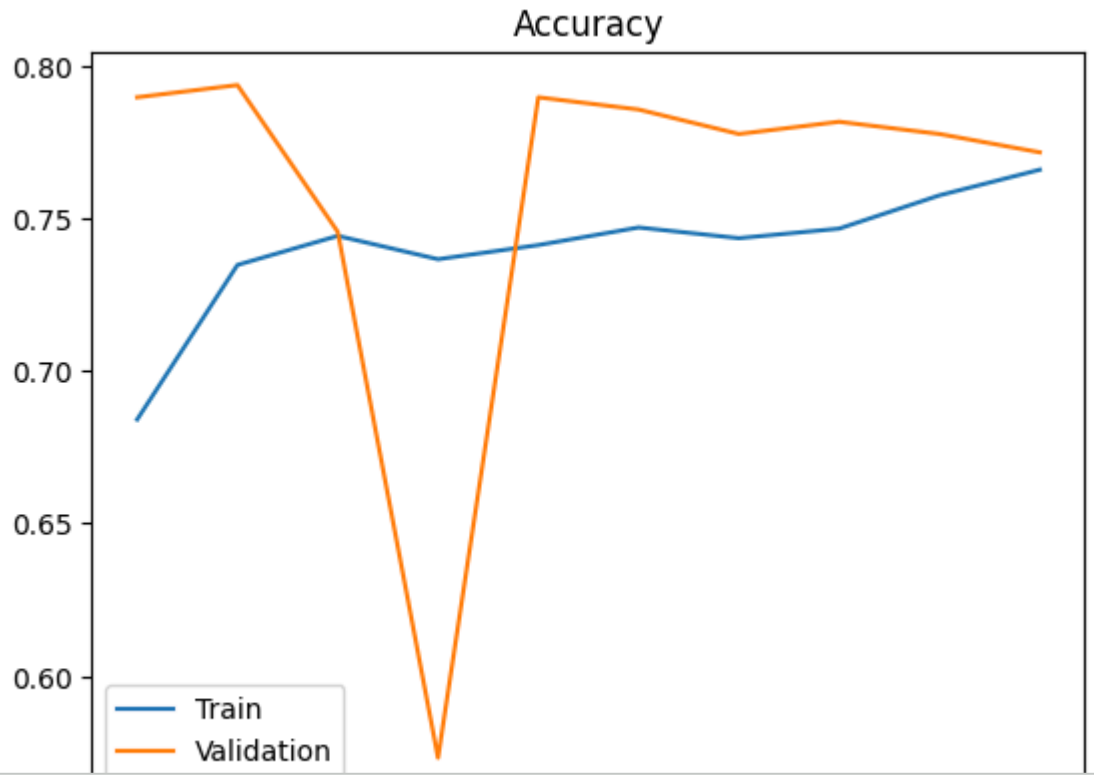
Total params: 3,304,769 (12.61 MB)
Trainable params: 3,304,769 (12.61 MB)
Non-trainable params: 0 (0.00 B)

```
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=10
)
```

```
Epoch 1/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 265s 1s/step - accuracy: 0.6840 - loss: 0.5755 -
Epoch 2/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 275s 1s/step - accuracy: 0.7346 - loss: 0.5215 -
Epoch 3/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 284s 1s/step - accuracy: 0.7442 - loss: 0.5114 -
Epoch 4/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 268s 1s/step - accuracy: 0.7365 - loss: 0.5131 -
Epoch 5/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 260s 1s/step - accuracy: 0.7412 - loss: 0.5076 -
Epoch 6/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 260s 1s/step - accuracy: 0.7469 - loss: 0.4936 -
Epoch 7/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 284s 1s/step - accuracy: 0.7434 - loss: 0.4932 -
Epoch 8/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 307s 1s/step - accuracy: 0.7466 - loss: 0.4944 -
Epoch 9/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 276s 1s/step - accuracy: 0.7575 - loss: 0.4668 -
Epoch 10/10
250/250 ━━━━━━━━━━━━━━━━━━━━ 263s 1s/step - accuracy: 0.7659 - loss: 0.4517 -
```

```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.legend(['Train','Validation'])
plt.title("Accuracy")
plt.show()
```




```
import numpy as np
from tensorflow.keras.preprocessing import image
```

```
def predict(img_path):
    img = image.load_img(img_path, target_size=(128,128))
    img = image.img_to_array(img)/255
    img = np.expand_dims(img, axis=0)

    pred = model.predict(img)

    if pred[0][0] > 0.5:
        print("🌿 Weed")
    else:
        print("🌱 Crop")
```

```
model.save("weed_model.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
```

```
from tensorflow.keras import models, layers

# Hyperparameters
learning_rate = 0.001
dropout_rate = 0.5

model = models.Sequential([
    layers.Conv2D(32,3,activation='relu',input_shape=(128,128,3)),
    layers.MaxPooling2D(),

    layers.Conv2D(64,3,activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128,3,activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(128,activation='relu'),

    layers.Dropout(dropout_rate),  # 🔥 NEW

    layers.Dense(1,activation='sigmoid')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_c
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
from tensorflow.keras.optimizers import Adam

optimizer = Adam(learning_rate=learning_rate)

model.compile(
    optimizer=optimizer,
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
```

```
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=20,
    callbacks=[early_stop]
)
```

```
Epoch 1/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 286s 1s/step - accuracy: 0.6638 - loss: 0.5904 -
Epoch 2/20
250/250 ━━━━━━━━━━━━━━━━━━━━ 265s 1s/step - accuracy: 0.7341 - loss: 0.5236 -
Epoch 3/20
187/250 ━━━━━━━━━━━━━━━━━━━━ 1:05 1s/step - accuracy: 0.7426 - loss: 0.5151
```

```
learning_rate = 0.001
dropout_rate = 0.5
batch_size = 32
```

```
learning_rate = 0.0001
dropout_rate = 0.3
batch_size = 64
```

```
learning_rate = 0.01
dropout_rate = 0.6
```

```
learning_rate = 0.0001
dropout_rate = 0.3
epochs = 20
```

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

train_data = train_data.map(lambda x, y: (data_augmentation(x), y))
```

```
#
```

```
model.evaluate(test_data)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 2s 287ms/step - accuracy: 0.8983 - loss: 0.2094
[0.20943677425384521, 0.8983050584793091]
```

Start coding or generate with AI.